

OCC: SOFTWARE ENGINEER YEAR 2

SAQA ID

NQF 6

Software design and testing C#

SDT621

PRACTICAL MODULE -PM-04

2026



Table of Contents

DECLARATION OF AUTHENTICITY	3
QUALIFICATION AND ASSESSMENT DETAILS	4
Submission Requirements	5
Evidence Collection Requirements	5
GitHub Submission Requirement.....	5
Example of Collecting Evidence for your submission pdf:	7
Payroll Testing and Debugging Project.....	8
Scenario: Mzansi Tech Contractors Payroll System	8
Business Rules.....	8
PM-04 Practical Requirement.....	9
Required Testing Levels for PM-04	9
1. Unit Testing.....	9
2. Integration Testing	9
3. System Testing	10
4. Retesting	10
5. Regression Testing.....	10
Validation Rules	10
PM-04 Marking Rubric	12
PM-04 Mapping: Mzansi Tech Contractors Payroll System	14

DECLARATION OF AUTHENTICITY

I Stern George (FULL NAME)

hereby declare that the contents of this assessment PM04

is entirely my own work, completed by me without any paraphrasing/copying, or presented as my own work accessed from any AI Apps, example ChatGPT, Co-pilot, Perplexity, or any other App.

I have read and understood the Examination Rules and Regulations.

Signature: _____

Date: 2026/05/15

QUALIFICATION AND ASSESSMENT DETAILS

Programme Title	OCC: SOFTWARE ENGINEER
Module Title	Software design and testing C#
Module Code	SDT621
NQF Level	6
Assessment Type	PRACTICAL MODULE (PM-O4)
Hand Out Date	04 / 0 5 / 2026
Hand in Date	15 / 05 / 2026
Total Marks	100
Project Lead Developer	Lusukama Selemani
Internal Moderator	Faith Muwishi
Internal Moderator	Michelle Du Toit
External Moderator	

Student Name and Surname	Stern George
Student Number	20240036

Instructions:

Please read the following instructions carefully before attempting this Practical Module (PM-04):

General Instructions

- ❖ Read each question carefully and take note of the mark allocation before answering.
- ❖ Ensure that all questions are answered.
- ❖ Provide clear, well-structured C# code and explanations where required.
- ❖ Apply your understanding of software testing and debugging concepts (PM-04) when solving the tasks.

Submission Requirements

For your final submission, you must include the following:

- ❖ Declaration of Authenticity (signed)
(If the declaration is already included in this document, you may complete and sign it within this document.)
- ❖ Completed Answer Sheet (PDF format)
(Preferably use this current document as your answer sheet.)
- ❖ Visual Studio Project Files
(Submit your complete project compressed as a ZIP file.)
- ❖ GitHub Repository Link
(Ensure your repository contains the full and correct solution.)

Evidence Collection Requirements

To support your submission, you must:

- ❖ Capture screenshots of the application output and testing results
- ❖ Ensure each screenshot displays the full screen, including system date and time
- ❖ Include evidence of:
 - Application running (GUI output)
 - Test results (MSTest / Test Explorer)
- ❖ Do NOT submit screenshots of source code
- ❖ Submit your complete project folder as a ZIP file

GitHub Submission Requirement

You are required to:

- ❖ Push your project code to GitHub
- ❖ Ensure the repository contains the correct and complete solution
- ❖ Share the GitHub repository link in your submission for review and verification
- ❖ Example: <https://github.com/your-username/project-name>

Testing Requirements (PM-04 Focus):

You are required to demonstrate:

- ❖ Unit Testing (MSTest for payroll calculations)
- ❖ Integration Testing (input → calculation → output)
- ❖ System Testing (complete application workflow)
- ❖ Error handling and validation testing
- ❖ Debugging and correction of defects

Test Report Requirement

You must produce a **Test Report** that includes:

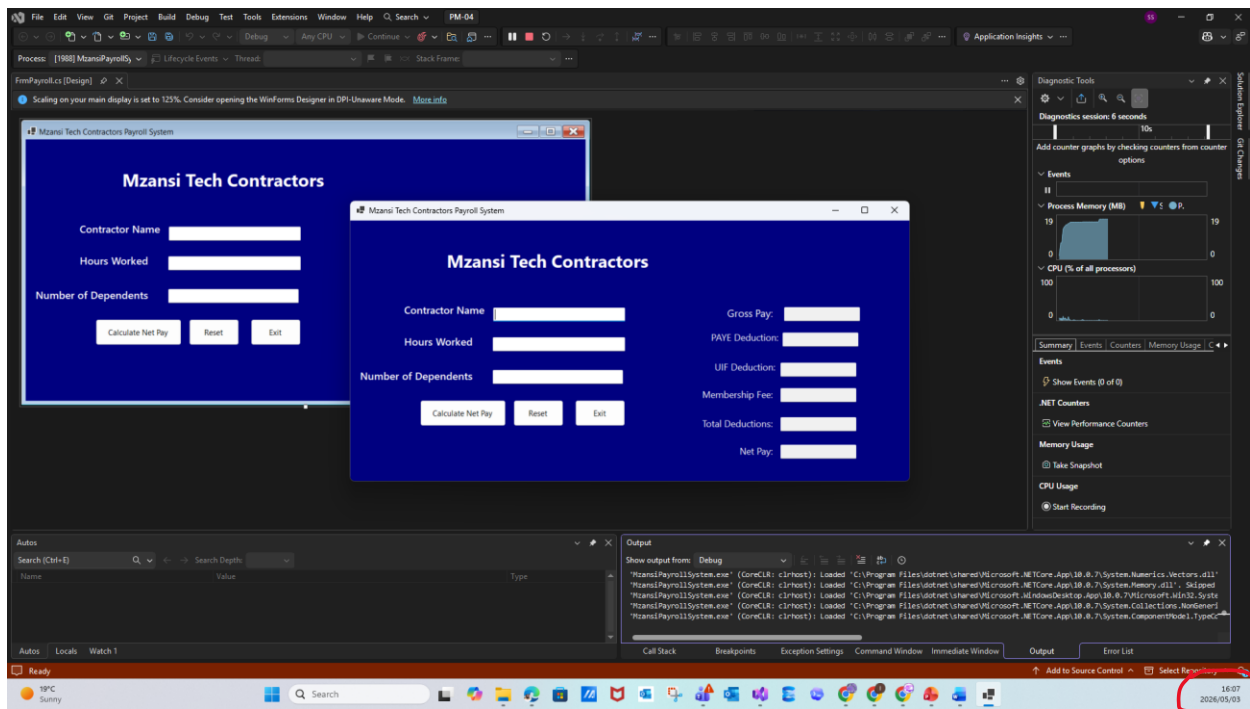
- ❖ Test cases
- ❖ Expected results
- ❖ Actual results
- ❖ Pass/Fail status
- ❖ Identified defects
- ❖ Corrections made
- ❖ Retesting results

Assessment Outcomes:

- ❖ IAC0102 – The correct tests are applied
- ❖ IAC0103 – A test report is produced
- ❖ PM-04 – Testing and debugging software systems

Example of Collecting Evidence for your submission pdf:

Mzansi Tech Contractors Payroll System Design :



Ensure each screenshot shows the **full screen**, including the **system date and time**

Payroll Testing and Debugging Project

Scenario: Mzansi Tech Contractors Payroll System

Mzansi Tech Contractors is a South African company that places temporary IT contractors, including systems analysts and software developers, with client companies. Contractors are paid in South African Rand (R) based on the number of hours worked.

The company uses a payroll application to calculate each contractor's gross pay, statutory deductions, membership deductions, and net pay. However, management has reported that the current system sometimes produces incorrect calculations and does not properly validate user input.

Business Rules

The application must accept:

- ❖ Contractor name
- ❖ Number of hours worked
- ❖ Number of dependents

The system must calculate:

Hourly rate: R950.00 per hour

Gross Pay = Hours Worked × Hourly Rate

Deductions:

UIF Deduction: 1% of gross pay, based on South African UIF rules where the employee contributes 1% and the employer contributes 1%.

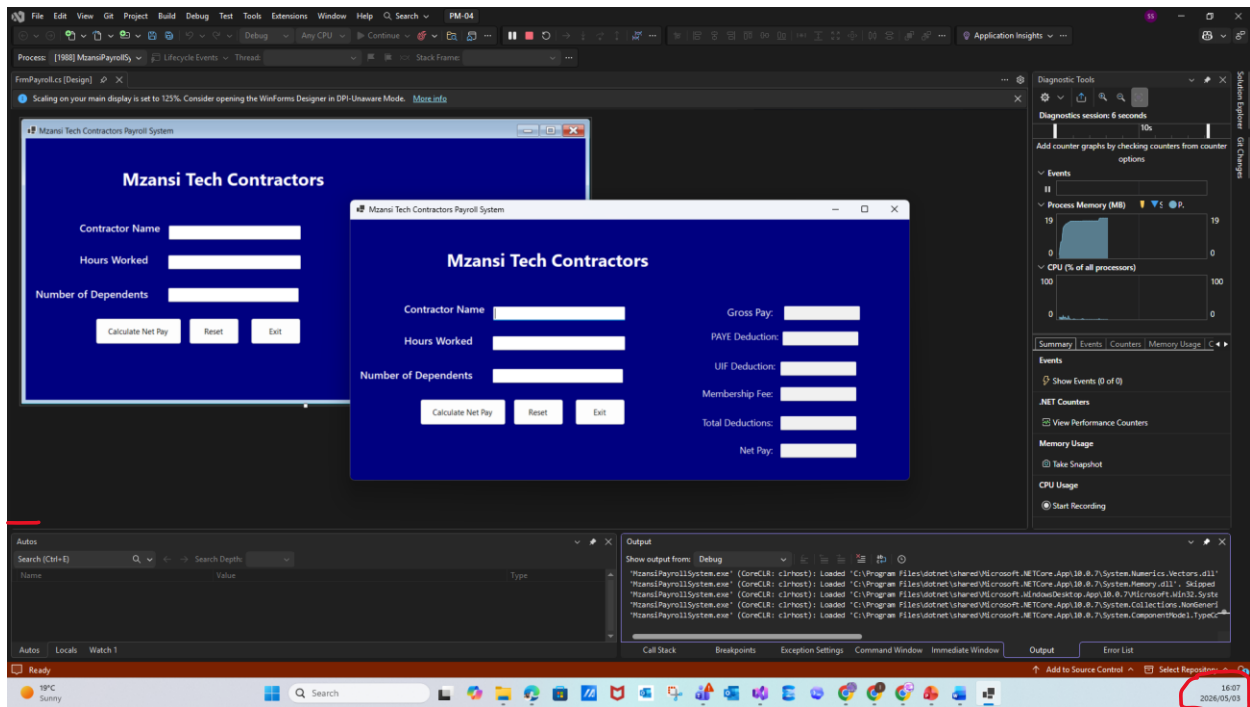
PAYE Deduction: Use a simplified SARS-based payroll rule for the assessment. SARS provides official employee tax deduction tables for PAYE calculations.

For this practical assessment, use:

- ❖ **PAYE** = (Gross Pay - (Gross Pay × 0.0575 × Number of Dependents)) × 25%
- ❖ **Membership Fee:** 13% of gross pay
- ❖ **Net Pay** = Gross Pay – UIF – PAYE – Membership Fee

PM-04 Practical Requirement

Design a Windows Forms GUI Application that allows the user to capture contractor payroll details and display:



- ❖ Gross pay
- ❖ UIF deduction
- ❖ PAYE deduction
- ❖ Membership fee
- ❖ Total deductions
- ❖ Net pay

Required Testing Levels for PM-04

Learners must perform the following testing levels:

1. Unit Testing

Test individual calculations separately.

Test Case	Input	Expected Output	Result
Gross Pay			
UIF			
Membership			
PAYE			
Net Pay			

2. Integration Testing

Test whether the input, calculation, and output sections work together correctly.

Examples:

Hours entered by the user are correctly used in the gross pay calculation.

Gross pay is correctly passed into UIF, PAYE, and membership calculations.

All deductions are correctly combined to calculate net pay.

3. System Testing

Test the full payroll application as one complete system.

Step	Action	Expected Result
1		
2		
3		
4		
5		

4. Retesting

After fixing identified defects, learners must run the same failed tests again to confirm that the system now works correctly.

5. Regression Testing

Learners must confirm that fixing one defect did not break another part of the payroll application.

Produce a short test report showing expected results, actual results, defects found, and corrections made

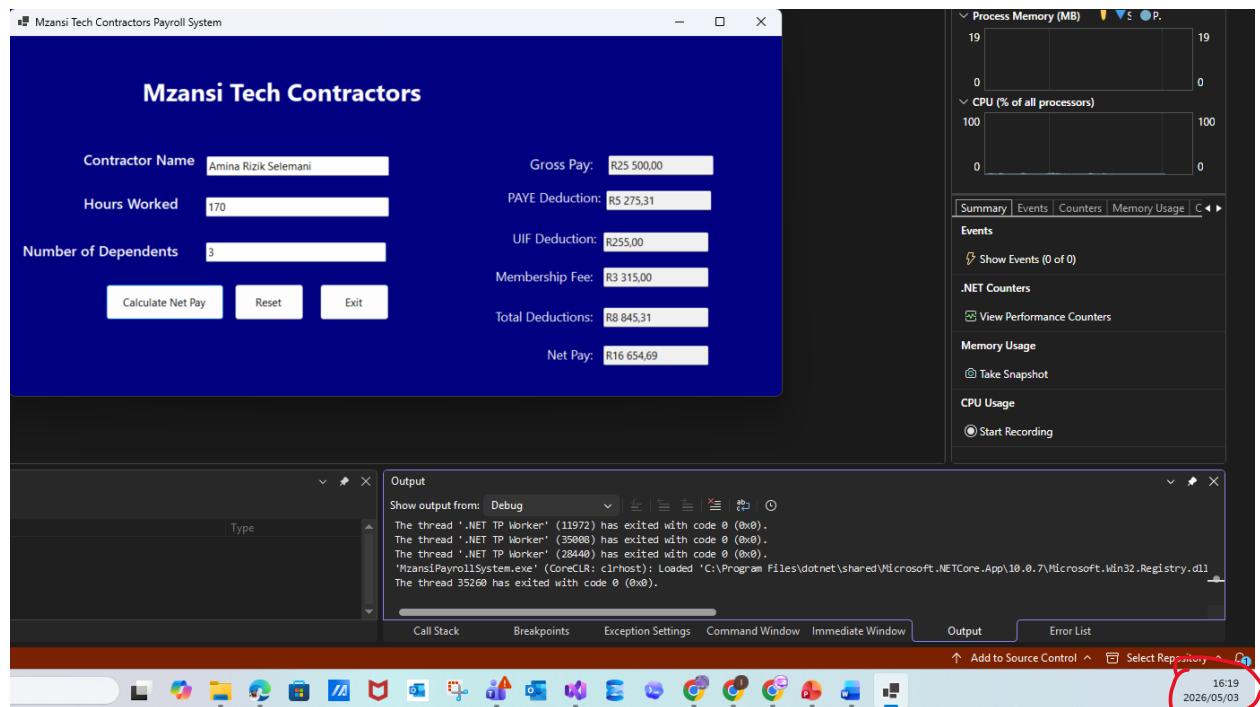
Validation Rules

The system must not allow:

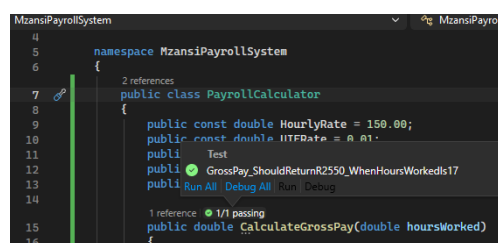
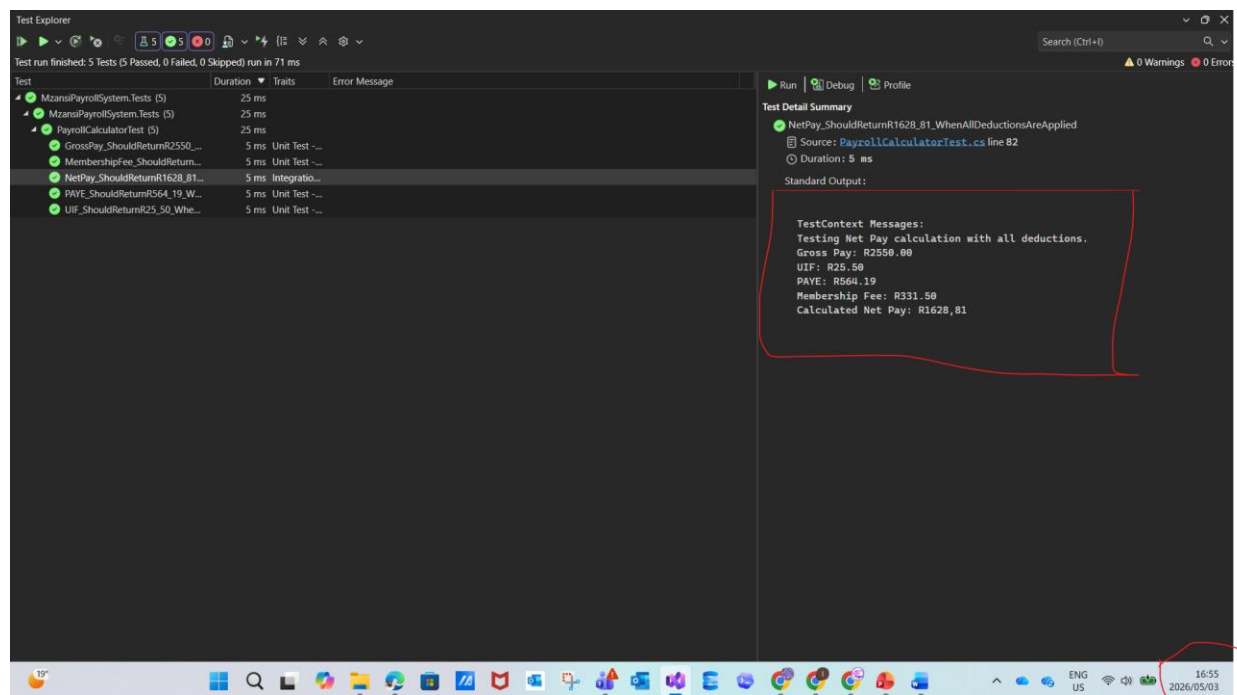
- ❖ Empty contractor name
- ❖ Negative hours worked
- ❖ Negative dependents
- ❖ Non-numeric input for hours or dependents
- ❖ Dependents greater than a reasonable limit, for example 10

END OF PAPER

Output Example



After test passed:



PM-04 Marking Rubric

Section	Criteria	Marks
User Interface Design (10)	Clear and professional layout	2
	Correct labels and input fields	2
	Proper alignment and spacing	2
	Appropriate use of controls	2
	Output section clearly structured	2
Input Validation (10)	Validates empty fields	2
	Validates numeric input	2
	Prevents negative values	2
	Validates dependents range (0–10)	2
	Displays user-friendly error messages	2
Payroll Calculations (20)	Correct Gross Pay calculation	4
	Correct UIF calculation (1%)	4
	Correct Membership Fee (13%)	4
	Correct PAYE calculation	4
	Correct Net Pay calculation	4
Application Functionality (10)	Calculate button works	3
	Reset button clears fields	3
	Exit button works	2
	System runs without errors	2
Unit Testing (20)	Test project created correctly	3
	PayrollCalculator class used	3
	Gross Pay test	3
	UIF test	3
	Membership Fee test	3
	PAYE test	3
	Net Pay test	2
Test Quality & Documentation (15)	Meaningful test names	3
	Use of TestCategory	3
	Use of TestContext logging	3
	Expected vs actual validation	3

	Professional structure	3
Integration & System Testing (10)	Integration testing explained	3
	System testing scenario	3
	End-to-end workflow tested	2
	Evidence provided (screenshots/logs)	2
Code Quality (5)	Clean, readable code	2
	Proper naming conventions	2
	Separation of logic (Calculator class)	1
TOTAL		100

PM-04 Mapping: Mzansi Tech Contractors Payroll System

PM-04 Requirement / Outcome	Project Activity	Evidence Required	Marks
Apply correct tests	Unit testing using MSTest for gross pay, UIF, PAYE, membership fee, and net pay	Test Explorer screenshot showing passed tests	15
Test system calculations	Verify all payroll formulas produce correct results	Test cases with expected vs actual results	15
Test integrated components	Confirm inputs, calculation class, and output textboxes work together	Screenshot of completed payroll form	10
Perform system testing	Test the full application from start to finish	System test table and screenshots	10
Identify defects	Record any errors found during testing, such as wrong formula or failed reset button	Defect log / bug report	10
Debug source code	Correct identified errors in the application	Before-and-after explanation or screenshots	10
Retest fixed defects	Run the same test again after fixing the issue	Retest evidence showing pass result	10
Produce test report	Document test cases, expected results, actual results, and pass/fail status	Completed test report	10
Follow coding standards	Use clear names, readable code, and separated PayrollCalculator class	Source code screenshots / GitHub link	5
Provide complete project evidence	Submit application, tests, screenshots, and report	Final zipped project / GitHub repository	5
Total			100